

Language Model Representations for Efficient Few-Shot Tabular Classification

Inwon Kang
Rensselaer Polytechnic Institute
Troy, New York, USA
kangi@rpi.edu

Parikshit Ram
IBM Research
Yorktown Heights, New York, USA
Parikshit.Ram@ibm.com

Yi Zhou
IBM Research
San Jose, California, USA
Yi.Zhou@ibm.com

Horst Samulowitz
IBM Research
Yorktown Heights, New York, USA
samulowitz@us.ibm.com

Oshani Seneviratne
Rensselaer Polytechnic Institute
Troy, New York, USA
senevo@rpi.edu

Abstract

The Web is a rich source of structured data in the form of tables, from product catalogs and knowledge bases to scientific datasets. However, the heterogeneity of the structure and semantics of these tables makes it challenging to build a unified method that can effectively leverage the information they contain. Meanwhile, Large language models (LLMs) are becoming an increasingly integral component of web infrastructure for tasks like semantic search. This raises a crucial question: can we leverage these already-deployed LLMs to classify structured data in web-native tables (e.g., product catalogs, knowledge base exports, scientific data portals), avoiding the need for specialized models or extensive retraining? This work investigates a lightweight paradigm, **Table Representation with Language Model (TaRL)**, for few-shot tabular classification that directly utilizes semantic embeddings of individual table rows. We first show that naive application of these embeddings underperforms compared to specialized tabular models. We then demonstrate that their potentials can be unlocked with two key techniques: removing the common component from all embeddings and calibrating the softmax temperature. We show that a simple meta-learner, trained on handcrafted features, can learn to predict an appropriate temperature. This approach achieves performance comparable to state-of-the-art models in low-data regimes ($k \leq 32$) of semantically-rich tables. Our findings demonstrate the viability of reusing existing LLM infrastructure for efficient semantics-driven pathway to reuse existing LLM infrastructure for Web table understanding.

CCS Concepts

• **Information systems** → **Document representation**; *Data encoding and canonicalization*; • **Applied computing** → *Document management and text processing*.

Keywords

structured data, tables, few-shot learning, large language models, semantic embeddings

ACM Reference Format:

Inwon Kang, Parikshit Ram, Yi Zhou, Horst Samulowitz, and Oshani Seneviratne. 2026. Language Model Representations for Efficient Few-Shot Tabular Classification. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792477>

1 Introduction

The Web contains a rich mixture of structured data in the form of tables – from product catalogs and knowledge bases to scientific datasets and business intelligence dashboards. These tables often carry rich semantic information through their column names, categorical values, and textual features. As large language models (LLMs) become increasingly embedded in web infrastructure for tasks like semantic search and content understanding, a natural question arises: *can we leverage these already-deployed LLM systems to understand and classify structured data, without requiring additional specialized models or extensive retraining?*

Traditional approaches to structured data classification [9, 25] have relied on specialized models like gradient boosted trees or neural architectures designed specifically for tabular data. While effective, these methods do not directly leverage the semantic understanding that LLMs have developed through pre-training on massive text corpora. Recent work has explored bridging this gap through two main approaches: using LLMs as feature extractors combined with traditional classifiers [24, 42], or fine-tuning LLMs on large corpora of tabular data [14, 19]. However, the first approach still requires training a downstream model, while the second demands significant computational resources (days of GPU time) and large training datasets—resources that may not be available in many practical web applications.

Moreover, existing methods that serialize entire tables for LLM processing incur prohibitive computational cost at inference time. For instance, serializing a few-shot task with just 3 columns and 2 examples can easily exceed 100 tokens, and the quadratic attention complexity of the transformer architecture makes this approach-wimpractical for real-world web-scale applications. This raises a key question: *can we leverage the semantic understanding already present in off-the-shelf LLM embeddings for structured data classification, without the overhead of specialized training or token-intensive serialization?*

We investigate this question through the lens of *lightweight semantic understanding* – organizations with already-deployed LLMs



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792477>

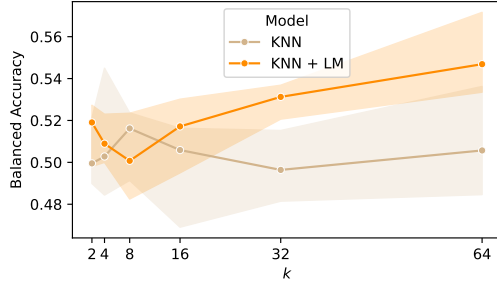


Figure 1: Comparison between KNN on raw data and on Llama 3.1 8B’s embeddings for k -shot classification on CARTE-CLF datasets. The results show that substituting raw features with LLM-derived embeddings alone yields a clear performance gain.

can reuse these systems for structured data tasks without additional infrastructure. This scenario is increasingly common: companies deploy LLMs for customer support, content generation, or semantic search, and subsequently encounter classification tasks on structured data (e.g., categorizing user-submitted forms, triaging support tickets with structured fields, or classifying product metadata). Rather than deploying a separate specialized model, can the existing LLM infrastructure be directly leveraged?

Our initial experiments reveal that LLM embeddings of individual table rows do contain useful predictive signal compared to raw feature values, as shown in Figure 1. However, a naive application of these embeddings performs poorly. In this work, we identify two critical factors that prevent off-the-shelf embeddings from reaching their potential: (1) the *geometry* of the embedding space, where embeddings exhibit anisotropy – i.e. not uniformly distributed in the latent space – and cluster in narrow regions, and (2) the need for *task-specific adaptation* to account for the varying semantic structure across different classification problems.

To address these challenges, we propose **Table Representation with Language Model (TaRL)**, a minimalist framework that makes LLM embeddings effective for few-shot tabular classification without any gradient-based training. Our approach requires only two lightweight adaptations: (1) a geometric correction (common component removal) to address embedding anisotropy, and (2) calibration of a single temperature parameter to adapt the classifier’s behavior to each task. This second adaptation is achieved through a simple meta-learned predictor trained on a small set of auxiliary datasets, requiring less than an hour of computation compared to days required by specialized tabular foundation models.

Our method is particularly effective for semantically-rich structured data—tables with meaningful column names, categorical values, and textual features common in web applications. On such datasets, TaRL achieves competitive performance with state-of-the-art specialized models in low-data regimes ($k \leq 32$), while providing up to 1000× speedup over autoregressive LLM inference. This makes it especially valuable for web-based applications where LLMs are already deployed and tables contain rich semantic information that can be leveraged for classification tasks.

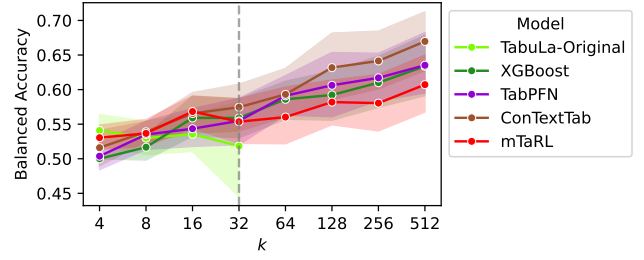


Figure 2: Comparison of mTaRL against SotA baselines on CARTE-CLF benchmark (11 datasets). mTaRL shows competitive performance against SotA models in lower k settings. Gray line indicates the context limit for TabuLa, and is able to scale much further than the 32-shot limit imposed for TabuLa.

Our work demonstrates that lightweight semantic understanding of structured data can be achieved by reusing existing LLM infrastructure, without requiring specialized architectures or extensive retraining. The key lies not in building more complex models, but in properly adapting the semantic representations that LLMs have already learned. Table 1 shows a comparison of our approach with existing tabular foundation models, highlighting our method’s unique advantages in terms of model agnosticism and minimal overhead.

Our contributions are as follows:

- We introduce TaRL, a lightweight framework that reuses off-the-shelf LLM semantic embeddings of serialized table rows for few-shot classification on web datasets, requiring no gradient-based training and minimal computational overhead.
- We identify two key semantic adaptations: Common Component Removal (CCR) to mitigate embedding anisotropy and per-task temperature calibration to enable dynamic attention over context examples, and show they unlock much of the performance latent in LLM embeddings.
- We establish an oracle upper bound for perfect per-task calibration and approximate it with a simple meta-learned predictor that trains in matters of minutes, enabling practical deployment.
- We demonstrate competitive accuracy and dramatic runtime gains (up to 1000× under caching) on semantically-rich datasets of CARTE-CLF benchmarks in the low-data regime ($k \leq 32$), highlighting where table semantics wins.

2 Related Work

2.1 Table Understanding and Semantic Embeddings

It has been shown that learning effective representations for tabular data can significantly improve performance on downstream tasks [4, 10, 17, 40]. Previous works have explored various methods of learning such embeddings, ranging from task-specific learned embeddings [23, 37, 41] to pre-training strategies [22, 32, 39]. The resulting embeddings are then often used as input features for the downstream task, such as retrieval [6, 7, 43] or classification [21, 27].

	Semantic Embeddings	No Downstream Training	No Task Prompt*	Model Agnostic	Table-Tuning Overhead [†]
TabLLM [19]	✓	✗	✗	✗	Not Reported [‡]
TabPFN [20]	✗	✓	✓	✗	2 weeks
ConTextTab [33]	✓	✓	✓	✗	4-10 days
TabuLa-8B [14]	✓	✓	✗	✗	6 days
TaRL (ours)	✓	✓	✓	✓	< 1 hour

Table 1: Comparison of our proposed approach (TaRL) against existing tabular foundation models. TabPFN and ContextTab are specialized architectures designed specifically for tabular prediction task, while TabuLa-8B is an LLM fine-tuned specifically for few-shot tabular classification. In contrast, TaRL uses off-the-shelf LLM embeddings without any task-specific training or prompting.

*: Some methods require adding additional task-specific context to the input (e.g. column names, feature descriptions, etc.) in addition to the context examples to achieve good performance.

†: The cost of pre-training for *tabular* tasks – only the table-training overhead is considered for TabuLa-8B.

‡: The authors report fine-tuning language models, but do not specify the exact runtime.

2.2 Classical and Specialized Tabular Models

For many years, Gradient Boosted Decision Trees (GBDTs), such as XGBoost [9] and LightGBM [25], have been the state-of-the-art for tabular data, particularly in industrial applications and data science competitions. While deep learning models like TabNet [2] and FT-Transformer [15] have demonstrated competitive performance, GBDTs remain a strong and widely used baseline.

More recently, the field has seen the emergence of specialized foundation models for tabular data. TabPFN [20] introduced a prior-fitted network that uses a transformer to approximate Bayesian inference, achieving strong performance on small to medium sized tables without task-specific fine-tuning. ConTextTab [33] extended this approach by incorporating semantic information from column names, further improving performance. These models, while powerful, are specialized architectures designed specifically for tabular data. In contrast, our work investigates whether it is possible to achieve strong performance without resorting to specialized models, by instead adapting the general-purpose representations from off-the-shelf LLMs.

2.3 Large Language Models for Tabular Data

The use of LLMs for tabular data has been explored through several paradigms. One common approach is to use LLMs to generate feature embeddings for tabular data, which are then fed into a separate downstream classifier. For instance, Zhang et al. [42] and Kasneci and Kasneci [24] have shown that adding LLM-generated embeddings as additional features for models like XGBoost can improve performance. Han et al. [18] shows that LLM engineered features are especially powerful in few-shot settings. However, this still requires training a separate, often complex, downstream model or employs sophisticated (and often token intensive) prompting techniques. Our work diverges from this by proposing an approach that directly uses the LLM embeddings for classification without any gradient-based training.

Another line of research focuses on adapting LLMs to tabular data through fine-tuning or in-context learning. TabLLM [19] demonstrated that LLMs like GPT-3 can perform in-context learning for tabular tasks with carefully designed prompts. LIFT [11] improved few-shot performance by fine-tuning on datasets retrieved based on a new task description. More recent work, such

as Yang et al. [38], Table-GPT [28] and TabuLa-8B [14], has focused on large-scale fine-tuning of LLMs on millions of tables to instill strong tabular reasoning capabilities. While these methods have achieved impressive results, they are computationally intensive and require significant training resources. On the other hand, model-reprogramming [8], which repurposes pre-trained models for new tasks without modifying the model weights, can become a more efficient alternative by changing how the input data is presented to the model. This is typically done by learning a mapping from the new task’s input space to the model’s original input space. While this approach has been explored in various domains, its application to tabular data remains limited. Our work aims to fill this gap by demonstrating that with minimal adaptations, off-the-shelf LLM embeddings can be effectively used for tabular classification tasks, providing a lightweight alternative to the more resource-intensive methods mentioned above.

2.4 Geometry of Language Model Representations

Language model embeddings have been widely studied, particularly in the context of natural language processing tasks. BehnamGhader et al. [5] show that decoder-only LLMs can be used to generate powerful text embeddings. However, it is well-established that embeddings from transformer-based language models exhibit anisotropy, meaning they occupy a narrow cone in the vector space [13]. This can degrade the performance of downstream tasks that rely on distance metrics like cosine similarity. A common technique to mitigate this is to post-process the embeddings, for example, by subtracting the common component [3, 29]. Our work builds on this understanding, but we go a step further by demonstrating that a simple, context-specific geometric correction is a critical component for unlocking the information contained in LLM embeddings for tabular data. While more sophisticated methods for correcting representation geometry exist, we find that a simple and targeted approach is sufficient to achieve a significant performance improvement.

3 Methodology

We hypothesize that two key factors can significantly unlock the potential of these embeddings: (1) a geometric correction to account

Table 2: Comparison of averaged runtime (in seconds) per k -shot task using TabuLa-8B with TaRL and original autoregressive way on CARTE-CLF benchmark.

k	TabuLa	TaRL(Ours)
2	36867.90	29.93 ($\times 1232$)
4	37518.86	42.19 ($\times 889$)
8	34785.27	67.19 ($\times 518$)
16	30369.63	118.06 ($\times 257$)
32	21455.61	222.76 ($\times 96$)

for the anisotropy of the embedding space, and (2) task-specific adaptation of the classifier to account for differences in data distributions across tasks. Our method, **TaRL**, avoids any gradient-based training to isolate the predictive power of the LLM embeddings themselves, as well any task-specific tuning. It consists of four main components: a serialization function to convert tabular rows into text, a kernel-based classifier that operates on the LLM embeddings, a geometric correction to the embedding space, and softmax-temperature scaling calibration to adapt to different tasks. We describe each component in detail below. We will use $\mathbf{x}$ to denote a row with d features, and $\mathbf{x} \in \mathbb{R}^{d_e}$ to denote the corresponding LLM embedding with d_e components.

3.1 From Tabular Rows to LLM Embeddings

To make tabular data accessible to a language model, we first serialize each row into a text sequence. We define a serialization function, $s = \text{serialize}(\mathbf{x})$, which converts a row $\mathbf{x} \in \mathbb{R}^{d_{\text{num}}} \times \mathbb{C}^{d_{\text{cat}}}$ into a human-readable string. This is done by concatenating the column names and their corresponding values. For example, a row with columns Feature A and Feature B and values 1.2 and 3.4 would be serialized as: Feature A: 1.2, Feature B: 3.4. Similarly, categorical features are encoded using their string representation (if available), and date/time features are converted into a standard format (YY-MM-DD:HH). Each row $\mathbf{x}$ is then embedded into a vector $\mathbf{x} \in \mathbb{R}^{d_e}$ using a pretrained, off-the-shelf LLM. It is worth noting that each row is embedded *separately* in TaRL. This is important to note because the cost of modern transformer-based embedding methods scales quadratically with the length of the input sequence [35]. Thus, unlike previous methods where the entire context and the task prompt was serialized into a single input, the cost of embedding for TaRL will only scale quadratically with respect to the *single longest context example*.

3.2 TaRL: Few-Shot Classification via Kernel Regression

We frame the few-shot classification problem as a kernel regression task. Given a support set of k labeled examples, $D_{\text{supp}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^k$ and set of m possible label classes $\mathcal{Y}$, where $\mathbf{x}_i$ is a tabular data point and $y_i \in \mathcal{Y}$ is its class label, our goal is to predict the label $\hat{y}_q$ for a new query point $\mathbf{x}_q$. We choose a kernel-based approach because it is non-parametric, makes few assumptions about the underlying data distribution, and is well-suited for the low-data regime of few-shot learning.

First, we compute the embeddings for the query and all support examples, yielding $\mathbf{x}_q$ and $\{\mathbf{x}_i\}_{i=1}^k$. Next, we compute the similarity $\text{sim}(\mathbf{x}_q, \mathbf{x}_i)$ between the query embedding $\mathbf{x}_q$ and each support embedding $\mathbf{x}_i$ using a similarity metric. We use the negative inverse cosine similarity, defined as: $\text{sim}(\mathbf{a}, \mathbf{b}) = -\cos^{-1}(\mathbf{a}, \mathbf{b})$, i.e. the angle between the two vectors. We opt to use this metric because it is bounded between $[-\pi, 0]$, which helps stabilize the softmax computation in the next step. These similarities are then normalized via a softmax function to produce a set of weights, w_i :

$$w_i = \frac{\exp(\text{sim}(\mathbf{x}_q, \mathbf{x}_i))}{\sum_{j=1}^k \exp(\text{sim}(\mathbf{x}_q, \mathbf{x}_j))} \quad (1)$$

Then, the query’s label embedding is computed as a weighted sum of the support set labels.

$$\hat{\mathbf{y}}_q = \sum_{i=1}^k w_i \mathbf{y}_i \quad (2)$$

Finally, the predicted class $\hat{y}_q$ is determined by comparing $\hat{\mathbf{y}}_q$ against the possible label class embeddings $\{\mathbf{y}_c\}_{c \in \mathcal{Y}}$, where $\mathbf{y}_c$ is the embedding of class c . The predicted class is the one with the highest similarity to $\hat{\mathbf{y}}_q$. It is worth noting here that it is also possible to use a one-hot vector of the labels in place of the embedding $\mathbf{y}_i$. We later examine the effect of both approaches.

$$\hat{y}_q = \arg \max_{c \in \mathcal{Y}} \text{sim}(\hat{\mathbf{y}}_q, \mathbf{y}_c) \quad (3)$$

This simple classifier allows us to directly probe the quality of the LLM embeddings without introducing any trainable parameters.

3.3 Geometric Correction with Common Component Removal (CCR)

As noted earlier, the geometry of the LLM embedding space can be a significant obstacle to good performance. Specifically, many language-embedding techniques are known to suffer from *anisotropy*, where the embeddings are not uniformly distributed in space, but rather clustered around a few dominant directions [13]. This can potentially lead to poor discrimination between different classes, as the embeddings may not be well-separated. In the context of our kernel regression classifier, this anisotropy can cause the similarity scores to be dominated by these common directions, leading to suboptimal weight distributions and ultimately poor classification performance. To address this, we apply a simple but effective geometric correction called Common Component Removal (CCR). This technique, adapted from work on sentence embeddings [3, 29], is designed to improve the isotropy of the embedding space by removing the average embedding vector (the common component) from all embeddings. We find that applying this technique (lines 6 and 7 of Algorithm 1) significantly improves performance across all datasets and few-shot settings.

3.4 Calibrating the Softmax Temperature

So far, the kernel classifier maps similarities directly to weights via a softmax function (Eq. 1). However, the scale of the similarities can vary significantly across different tasks and datasets – e.g., in some tasks, the similarities may be tightly clustered, while in others they may be more spread out. To account for this variability, we

introduce a tunable parameter γ to control the sharpness of the softmax distribution.

$$w_i = \frac{\exp(\gamma \cdot \text{sim}(\mathbf{x}_q, \mathbf{x}_i))}{\sum_{j=1}^k \exp(\gamma \cdot \text{sim}(\mathbf{x}_q, \mathbf{x}_j))} \quad (4)$$

This parameter γ can be interpreted as an inverse of the temperature parameter that adjusts the confidence of the weight distribution. Thus, a higher value of γ leads to a more *peaked* distribution, essentially mimicking a nearest-neighbor classifier, while a lower value leads to a more *uniform* distribution, effectively averaging over more support examples. In the following section, we show the careful tuning of γ allows the classifier to leverage the LM embeddings more effectively. Algorithm 1 summarizes the full **TaRL** prediction procedure, which combines all the components described above.

A heuristic for setting γ . In practice, we find that the optimal value of γ varies significantly across datasets and few-shot settings. This means that we need a way to select γ by just looking at the support set. One such heuristic is to set γ based on the *how similar the most similar support example is* – i.e., the maximum similarity between the query and any support example.

$$\hat{\gamma} = \frac{1}{\max(-\text{sim}(\mathbf{x}_q, \mathbf{x}_i))} \quad (5)$$

If the similarity of the closest example is very high (i.e., the closest example is a near-duplicate of the query), $\hat{\gamma}$ will approach infinity, meaning Eq. 4 effectively puts all the weight on the nearest-neighbor, and thus the final classification matches that of a one-nearest-neighbor classifier. On the other hand, as the similarity between the query and the closest example decreases, $\hat{\gamma}$ will cause the weight to be more evenly distributed across every sample in the context.

Algorithm 1 TaRL Few-Shot Prediction

- 1: **Input:** Support set $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^k$, Query row $\mathbf{x}_q$, Temperature γ , Label classes $\mathcal{Y}$
 - 2: $\mathbf{x}_i \leftarrow \text{LM}(\text{SERIALIZE}(\mathbf{x}_i))$; $\mathbf{y}_i \leftarrow \text{LM}(y_i)$ for $i \in \{1..k\}$
 - 3: $\mathbf{x}_q \leftarrow \text{LM}(\text{SERIALIZE}(\mathbf{x}_q))$
 - 4: $\mathbf{y}_c \leftarrow \text{LM}(c)$ for $c \in \mathcal{Y}$
 - 5: $\bar{\mathbf{x}} \leftarrow \frac{1}{k+1} \left((\sum_{i=1}^k \mathbf{x}_i) + \mathbf{x}_q \right)$
 - 6: $\mathbf{x}'_i \leftarrow \mathbf{x}_i - \bar{\mathbf{x}}$ for $i \in \{1..k\}$ # Apply CCR to support
 - 7: $\mathbf{x}'_q \leftarrow \mathbf{x}_q - \bar{\mathbf{x}}$ # Apply CCR to query
 - 8: Compute weights $\{w_i\}_{i=1}^k$ for $(\mathbf{x}'_q, \{\mathbf{x}'_i\}_{i=1}^k)$ using Eq. 4.
 - 9: $\hat{\mathbf{y}}_q \leftarrow \sum_{i=1}^k w_i \mathbf{y}_{y_i}$, where $\mathbf{y}_{y_i}$ is the embedding for class label y_i .
 - 10: $\hat{y}_q \leftarrow \arg \max_{c \in \mathcal{Y}} \text{sim}(\hat{\mathbf{y}}_q, \mathbf{y}_c)$
 - 11: **Return:** Predicted label $\hat{y}_q$
-

3.5 mTaRL: Learning to calibrate γ

While the heuristic-based method provides a simple way to set γ , it is not guaranteed to be optimal. To further improve performance, we propose a prototype meta-learning approach, mTaRL, that learns to predict the optimal γ from the properties of the support set itself. We first define an *oracle* classifier: a classifier that uses the optimal γ for each task, where the optimal γ is chosen by minimizing the cross-entropy loss on the query’s true label. This oracle classifier serves as an upper bound on the performance of our method, as

it represents the best possible performance we could achieve if we knew the optimal γ for each task. We then approximate this oracle by training a simple regression model on a set of general, dataset-agnostic meta-features. These features are not engineered for any specific dataset but rather capture only the task-relevant context-query statistics of the embedding space, such as mean similarity, variance, and class imbalance (see Appendix A.4 for details). This lightweight, support-set-level feature extraction allows for rapid, gradient-free adaptation to new tasks and stands in stark contrast to computationally intensive alternatives like LLM fine-tuning. The model is trained on datasets with known optimal γ values, determined via cross-validation.

Given a new few-shot task, we compute the same summary statistics from its support set and use the trained regression model to predict the optimal γ . This approach allows us to adaptively select γ for each new task without any gradient-based training on the task itself. We find that this meta-learning approach consistently outperforms the heuristic method, demonstrating the effectiveness of learning to adapt the classifier to different tasks.

Algorithms 2 and 3 summarize the meta-training and meta-inference procedures, respectively. The meta-training procedure constructs a dataset of support set features and optimal γ values from multiple datasets and few-shot settings. The optimal γ is chosen by picking the γ that maximizes the probability of the correct prediction on a held-out query point. This is done for multiple tasks sampled from each dataset to create a diverse meta-training set. The features are extracted *only* from the support set using a feature engineering function f . More detail on f can be found in Appendix A.4. The meta-inference procedure trains a regression model on this meta-dataset and uses it to predict $\bar{\gamma}$ for new tasks. This allows us to adaptively select γ for each new task without any gradient-based training on the task itself.

4 Experiments

4.1 Datasets

We evaluate our method on a suite of datasets selected from the CARTE benchmark [26] and TabArena benchmark [12].

The CARTE benchmark, consisting of 11 binary classification and 40 regression tasks, is curated with a focus on datasets that have *low numerical features* and *high-cardinality textual features*, making it well suited for evaluating tabular models that can handle semantic content. Additionally, the datasets in this benchmark have meaningful column names, which are important for LLMs to effectively utilize the tabular data.

TabArena is a more recent benchmark that includes 32 binary, 6 multiclass classification tasks¹ and 13 regression tasks, with a focus on datasets that best represent the most *general* tabular classification task. Unlike CARTE, datasets in TabArena may not have meaningful column names and can include a higher proportion of numerical features.

A detailed report on the dataset characteristics can be found in Table 6 in Appendix A.4. In total, we evaluate our approach using 45 classification datasets and 53 regression datasets. Our main results focus on the binary classification datasets of each benchmark

¹We discard 4 classification datasets due to issues with large memory usage.

as TaRL is a classification mechanism. Additionally, we also report the results on the multiclass classification datasets from TabArena and binned versions of the regression datasets in Appendix A.3. We use all 98 of the datasets for generating the meta-dataset for training the meta-learner.

4.2 Sampling Procedure

To simulate a few-shot learning scenario, we randomly sample a small number of examples from the training set to form the support set. We vary the size of the support set, k , from 2 to 512, using the following values: $k \in \{2, 4, 8, 16, 32, 64, 128, 256, 512\}$. To ensure a fair comparison, we maintain the class balance of the original dataset when sampling the support set. For each value of k , we repeat the experiment with 128 different randomly sample context-query pairs. It is worth noting that the baseline few-shot results reported in Gardner et al. [14] were obtained by optimizing the context and re-using it for every query. However, this approach makes the assumption that *we can choose the best possible context for each task*, which is not realistic in practice. Thus, in order to simulate a realistic low-data scenario, we opt to re-sample the support set for each query, which makes the task more challenging and realistic. All the results reported in this work are based on this sampling procedure for a fair comparison.

4.3 Models and Baselines

We compare our method against three categories of baselines:

- **Traditional Tabular Learners:** We include standard, widely-used models for tabular data: XGBoost [9] and k-Nearest Neighbors (KNN). These models are trained on the support set and evaluated on the corresponding query row. Since the context size is quite small, we do not apply any hyperparameter optimization, aside from setting the number of neighbors in KNN to $\log_2(k)$.
- **Specialized Tabular Foundation Models:** We compare against state-of-the-art foundation models designed specifically for tabular data: TabPFN [20], and ConTextTab [33], a semantics-enhanced version of the PFN backbone. These models are used off-the-shelf without any fine-tuning.
- **LLM-based Models:** We compare against other methods that use LLMs for tabular data. This includes TabuLa-8B [14], Llama 3.1 8B variants [1], Granite 3 8B [16] and Qwen 3 8B [34]. In addition, we also test a smaller encoder-only model from Sentence Transformers miniLM-L6-v2 [31].

For picking the *optimal* performance and constructing the meta-dataset, we choose values of γ from $2^{[-10, 15]}$. We also consider using the LLM in an auto-regressive manner as used by previous works [14, 19] to directly perform few-shot classification via in-context prompting. However, as Table 2 shows, the prohibitive resource usage of this approach made it difficult to run on all datasets. Thus, for TabuLa-8B, we opt to spend our available computing budget on CARTE-CLF for $k \leq 32$, whose semantic content should be well-suited for LLMs and would provide a fair baseline². In addition, we also use the embeddings of TabuLa-8B for TaRL unless otherwise specified.

²Gathering all the results for TabuLa-8B took approximately 12 days on our setup reported in Appendix A.1.1.

Evaluation Metric. In order to account for true performance with respect to label imbalance, we use balanced accuracy as our primary evaluation metric. Balanced accuracy is defined as the average of recall obtained on each class. This metric is particularly important in few-shot settings where class imbalance can significantly skew results.

5 Results and Analysis

The main results are summarized in Table 3, which compares our proposed meta-learning method (mTaRL) against several baselines across different benchmarks and varying numbers of shots (k). First, we see that the oracle classifier yields extremely strong performance, achieving the best or near-best results across all benchmarks and k values. It is important to note that this performance is not necessarily an achievable upper bound as it relies on an oracle to select the best γ for each instance. However, it demonstrates the potential of LLM embeddings when properly adapted.

5.1 Predictive Signal in Raw LLM Embeddings

Our first analysis is to examine whether the raw LLM embeddings contain any useful signal for tabular classification. While previous works have examined using LLM embeddings as *additional* features to the tabular data, we are interested in whether they can be used *directly* for classification without any additional features or training. To this end, we compare a simple KNN classifier using the raw tabular data against a KNN classifier using the LLM embeddings from TabuLa-8B using $\log_2(k)$ as the neighbor size for each k -shot task. Figure 1 shows the results on CARTE-CLF benchmark. We find that that 1). KNN on raw data performs poorly in low k settings, and 2). KNN on LLM embeddings performs significantly better, demonstrating that the LLM embeddings do indeed contain useful information for tabular classification. However, the performance is still far from satisfactory, indicating that further adaptation is necessary.

Another interesting aspect to consider when using LLM embeddings is how to best represent the class labels. A simple approach is to use one-hot encoded vectors for each class, but this does not leverage the semantic information contained in the label text. Instead, we can use the LLM to generate embeddings for each label, which can then be used in conjunction with the instance embeddings. As discussed in Section 3.2, TaRL uses the embedding of the label to compute a weighted sum of the context labels to make a final prediction. However, it is also possible to use w_i directly as weights for the one-hot encoded label of each context – i.e. a weighted voting classifier. To determine whether using label embeddings provides an advantage, we compare the performance of the two versions as in Table 4. We find that using label embeddings significantly improves the performance of the oracle classifier over all k values, showing that using the label embeddings instead of one-hot encodings provides a meaningful advantage.

5.2 Impact of LLM choice

Having confirmed that the predictive signal is present in the raw LLM embeddings, we investigate the *impact* of using different LLM backbones. We compare the performance of the oracle classifier

Table 3: Detailed comparison of average balanced accuracy score per benchmark. Best performance in each benchmark/ k combination is marked with bold and the second best with underline. CB: CARTE-Binary, TB: TabArena-Binary. The values inside parentheses indicate the number of datasets per benchmark.

Benchmark	Model	Rank		k (number of shots)							
		$k \leq 32$	$k > 32$	4	8	16	32	64	128	256	512
CARTE Binary (11)	KNN	8.0568	7.7614	0.5085	0.5094	0.5026	0.4993	0.4965	0.5056	0.5179	0.5096
	KNN-Emb.	7.3977	7.2955	0.5125	0.5216	0.5317	0.5517	0.5450	0.5612	0.5645	0.5879
	XGBoost	6.8295	6.1250	0.5000	0.5167	0.5590	<u>0.5590</u>	0.5860	0.5922	0.6100	0.6338
	TabPFN	6.9318	<u>5.6364</u>	0.5038	0.5350	0.5434	0.5548	<u>0.5910</u>	<u>0.6062</u>	<u>0.6170</u>	<u>0.6352</u>
	ConTextTab	6.3295	5.0795	0.5160	0.5390	<u>0.5655</u>	0.5745	0.5931	0.6318	0.6415	0.6697
	TabuLa-Original	6.5114	N/A	0.5409	0.5300	0.5354	0.5185	N/A	N/A	N/A	N/A
	mTaRL	<u>6.4886</u>	6.6705	<u>0.5305</u>	<u>0.5367</u>	0.5683	0.5535	0.5603	0.5819	0.5804	0.6072
	Oracle	0.5568	1.0000	0.7742	0.8057	0.8220	0.8243	0.8260	0.8469	0.8481	0.8335
TabArena Binary (28)	KNN	6.3973	7.4866	0.5355	0.5663	0.6160	0.6072	0.6081	0.6347	0.6393	0.6216
	KNN-Emb.	7.5089	7.6875	0.5250	0.5495	0.5359	0.5573	0.6138	0.6230	0.6524	0.6349
	XGBoost	6.1071	5.8036	0.5179	0.5770	0.6136	0.6678	0.6839	0.7003	0.7153	0.7240
	TabPFN	4.7545	4.1875	<u>0.5688</u>	0.6126	0.6507	0.6961	0.7141	0.7499	<u>0.7700</u>	<u>0.7526</u>
	ConTextTab	<u>5.4152</u>	<u>4.5268</u>	0.5691	<u>0.5944</u>	<u>0.6352</u>	<u>0.6747</u>	<u>0.7008</u>	<u>0.7363</u>	0.7726	0.7544
	mTaRL	6.7411	7.3438	0.5475	0.5712	0.5744	0.5974	0.6263	0.6311	0.6409	0.6416
	Oracle	0.5312	1.1071	0.8038	0.8197	0.8397	0.8551	0.8776	0.8862	0.8932	0.8741

Table 4: Comparison of average balanced accuracy score when using label embeddings (Oracle) vs. one-hot encoded labels (Oracle-V).

Model	k (number of shots)							
	4	8	16	32	64	128	256	512
Oracle	0.7742	0.8057	0.8220	0.8243	0.8260	0.8469	0.8481	0.8335
Oracle-V	0.5869	0.6360	0.6884	0.7041	0.7490	0.7703	0.7882	0.7892

using embeddings from different LLMs, including TabuLa-8B, Granite3.3 (Base and Instruction-tuned), Llama3.1-8B, Qwen3 (Base, Instruction-tuned, and Embedding-tuned), and a smaller encoder-only model from Sentence Transformers (all-MiniLM-L6-v2). The results can be seen in in Table 5. We find that while all models perform reasonably well, there is a significant variation in performance across different models. Interestingly, we observe that MiniLM, which has only ~ 22 M parameters, shows a stronger performance compared to the much larger Qwen models, especially notable in lower k values. Another observation is that embedding-tuning significantly improves the performance of the Qwen model variants, although Llama and TabuLa show the strongest performance overall. In particular, we find that Llama3.1-8B shows generally the strongest performance, but it’s table-fine-tuned variant, TabuLa-8B, also shows very strong performance, especially in the lower k regimes.

5.3 The Power of γ Calibration

We then construct the most simple classifier as described in Section 3.2, which we refer to as the *naive* baseline. As shown in Figure 3, this naive approach performs poorly, barely above random chance. This confirms that a straightforward application of LLM embeddings is insufficient for tabular classification. However, it also raises the question of whether the poor performance is due to

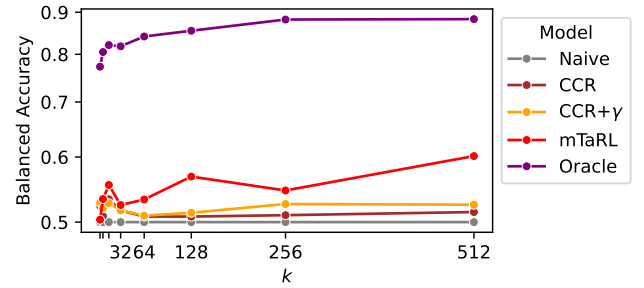


Figure 3: Balanced accuracy of classifiers built from LLM embeddings as more components are added.

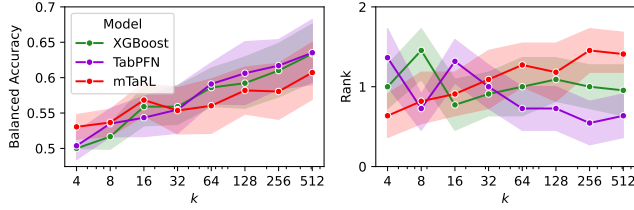
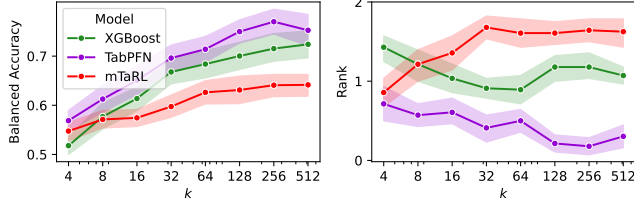
a fundamental lack of signal in the embeddings, or whether it is due to a failure to properly adapt the classifier to the task at hand. The results of Figure 1 suggest that the embeddings do contain useful information which can be extracted to some degree by a KNN classifier, suggesting that the *naive* kernel classifier alone is not sufficient to unlock the information contained in the embeddings. The results of applying the CCR and adjustment γ confirm this hypothesis. As can be seen in Figure 3, applying CCR and γ adjustments shows a boost in performance. However, even with both components, the performance is still far from satisfactory, indicating that further adaptation is necessary. Our prototype meta learning approach shows that it is indeed possible to further close the gap to the oracle classifier, again demonstrating that task-specific adaptation is crucial for good performance.

5.4 Semantic vs. Non-Semantic Datasets

An advantage of using LLM-based methods on tabular data is in the ability to leverage the semantic content of the tables, such as column names and values. To better understand the impact of semantic

Table 5: Comparison of average balanced accuracy score across different language models.

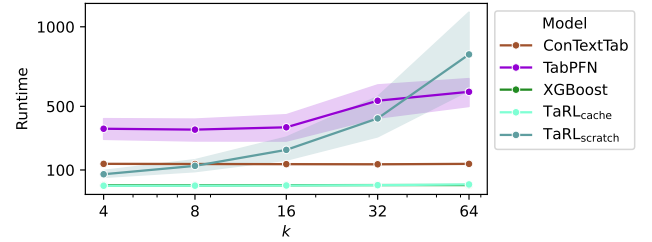
Model	k (number of shots)								
	2	4	8	16	32	64	128	256	512
Granite3.3-Base	74.27	<u>77.36</u>	78.43	77.28	78.57	77.85	80.42	79.51	80.52
Granite3.3-Inst.	73.16	74.94	79.56	80.53	80.54	<u>83.52</u>	81.84	84.27	<u>84.36</u>
Llama3.1-8B	<u>77.18</u>	76.90	80.12	<u>82.01</u>	83.76	84.68	86.07	85.40	86.32
Qwen3	57.43	56.77	55.42	56.45	54.92	56.95	56.81	56.48	55.66
Qwen3-Base	61.53	61.75	62.72	63.29	64.30	64.81	64.54	65.04	64.68
Qwen3-Emb.	73.90	75.82	75.69	78.11	77.01	78.14	77.72	78.43	79.10
all-MiniLM-L6-v2	74.24	75.26	76.21	77.84	76.48	77.57	76.14	77.44	77.29
TabuLa-8B	77.19	77.42	80.57	82.20	<u>82.43</u>	82.60	<u>84.69</u>	<u>84.81</u>	83.35

**(a) CARTE-Binary benchmark.****(b) TabArena-Binary benchmark.****Figure 4: Comparison of mTaRL against non-semantic baselines on (a) CARTE-CLF and (b) TabArena-CLF benchmarks. Higher is better for balanced accuracy (left) and lower is better for rank (right).**

content in the downstream performance, we separately analyze the performance of non-semantic baselines (XGboost, TabPFN) against mTaRL on two groups of datasets: CARTE-CLF, which contains rich semantic information, and TabArena-CLF, which contains mostly non-semantic data. As shown in Section 5.4, we find that mTaRL outperforms the non-semantic baselines on CARTE-CLF in $k \leq 32$, while non-semantic baselines show strong performance on TabArena-CLF. This suggests that the semantic content of the tables does indeed play a significant role in the effectiveness of LLM-based methods for tabular classification.

5.5 Runtime

An important consideration when deploying these models in real-life is the runtime performance. We first show in Table 2 that our proposed method reduces the runtime by as much as 1,000 times while showing stronger performance than using TabuLa. Figure 5 further breaks down the runtime performance of different models

**Figure 5: Comparison of runtime (in seconds) for different models across various k values on CARTE-CLF benchmark. TabuLa-8B is excluded because its runtimes are significantly higher (up to 1,000 times) than the other models.**

across various k values on CARTE-CLF benchmark. For a fair comparison, we evaluate two variants of TaRL: Embed every sample from scratch ($\text{TaRL}_{\text{scratch}}$), or using a caching scheme to cache all examples ahead of time ($\text{TaRL}_{\text{cache}}$). We see that for up to $k \leq 8$, even the $\text{TaRL}_{\text{scratch}}$ variant is quite fast, placing above XGBoost. However, as context size grows, the runtime of $\text{TaRL}_{\text{scratch}}$ increases significantly due to the repeated embedding computations. In contrast, $\text{TaRL}_{\text{cache}}$ remains extremely fast across all k values, as the only computation necessary is to calculate the support set weights and performing the predictions. This suggests that in a real-world scenario where an LLM is already deployed and the embeddings can be cached, our method can provide an extremely fast and efficient solution for few-shot tabular classification.

6 Conclusion

In this work, we show that web-native, semantically rich table data can be classified few-shot by reusing off-the-shelf LLM embeddings, using semantic adaptations: (1) common-component removal and (2) task-wise temperature calibration; closing much of the gap to specialized tabular FMs while being orders-of-magnitude cheaper. Our experiments demonstrate that mTaRL achieves comparable performance to specialized state-of-the-art tabular models in low-data regimes, especially when the datasets contain semantic content. We also show that this approach is highly efficient in computation while achieving stronger performance when compared to using an LLM directly for in-context learning, making TaRL a promising alternative to the expensive auto-regressive LLM inference.

References

- [1] Meta AI. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [2] Sercan Ö Arik and Tomas Pfister. 2021. TabNet: Attentive Interpretable Tabular Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 6679–6687. Issue 8. doi:10.1609/aaai.v35i8.16826
- [3] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. A Simple but Tough-to-Beat Baseline for Sentence Embeddings. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=SyK00v5xx>
- [4] Gilbert Badaro, Mohammed Saeed, and Paolo Papotti. 2023. Transformers for Tabular Data Representation: A Survey of Models and Applications. *Transactions of the Association for Computational Linguistics* 11 (2023), 227–249. doi:10.1162/tacl_a_00544
- [5] Parishad BehnamGhadar, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=IW1PR7vEBf>
- [6] Alberto Berenguer, Jose-Norberto Mazón, and David Tomás. 2024. Word Embeddings for Retrieving Tabular Data from Research Publications. *Machine Learning* 113, 4 (2024), 2227–2248. doi:10.1007/s10994-023-06472-0
- [7] Riccardo Cappuzzo, Paolo Papotti, and Saravanan Thirumuruganathan. 2020. Creating Embeddings of Heterogeneous Relational Datasets for Data Integration Tasks. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1335–1349. doi:10.1145/3318464.3389742
- [8] Pin-Yu Chen. 2022. Model Reprogramming: Resource-Efficient Cross-Domain Machine Learning. *arXiv preprint arXiv:2022.10629* (2022).
- [9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (New York, NY, USA) (KDD '16)*. Association for Computing Machinery, 785–794. doi:10.1145/2939672.2939785
- [10] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. 2020. TURL: Table Understanding through Representation Learning. *Proc. VLDB Endow* 14, 3 (2020), 307–319. doi:10.14778/3430915.3430921
- [11] Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. 2022. LIFT: Language-Interfaced Fine-Tuning for Non-language Machine Learning Tasks. In *Advances in Neural Information Processing Systems*, Vol. 35. 11763–11784. https://proceedings.nips.cc/paper_files/paper/2022/hash/4ce7fe1d2730f53cb3857032952cd1b8-Abstract-Conference.html
- [12] Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Praatek Mutalik Desai, David Salinas, and Frank Hutter. 2025. TabArena: A Living Benchmark for Machine Learning on Tabular Data. *arXiv preprint arXiv:2506.16791* (2025).
- [13] Kavin Ethayarajh. 2019. How Contextual Are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (Hong Kong, China), Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, 55–65. doi:10.18653/v1/D19-1006
- [14] Josh Gardner, Juan C. Perdomo, and Ludwig Schmidt. 2024. Large Scale Transfer Learning for Tabular Data via Language Modeling. *arXiv preprint arXiv:2406.12031* (2024). <http://arxiv.org/abs/2406.12031>
- [15] Yury Gorishniy, Ivan Rubachev, Valentin Khruikov, and Artem Babenko. 2021. Revisiting Deep Learning Models for Tabular Data. In *Advances in Neural Information Processing Systems*, Vol. 34. 18932–18943.
- [16] IBM Granite Team. 2024. Granite 3.0 Language Models. <https://github.com/ibm-granite/granite-3.0-language-models>
- [17] Cheng Guo and Felix Berkhahn. 2016. Entity Embeddings of Categorical Variables. *arXiv preprint arXiv:1604.06737* (2016).
- [18] Sungwon Han, Jinsung Yoon, Sercan Ö. Arik, and Tomas Pfister. 2024. Large Language Models Can Automatically Engineer Features for Few-Shot Tabular Learning. In *Proceedings of the 41st International Conference on Machine Learning*. PMLR, 17454–17479. <https://proceedings.mlr.press/v235/han24f.html>
- [19] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. TabLLM: Few-shot Classification of Tabular Data with Large Language Models. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*. PMLR, 5549–5581. <https://proceedings.mlr.press/v206/hegselmann23a.html>
- [20] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. 2025. Accurate Predictions on Small Data with a Tabular Foundation Model. *Nature* 637, 8045 (2025), 319–326. doi:10.1038/s41586-024-08328-6
- [21] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. *arXiv preprint arXiv:2012.06678* (2020).
- [22] Hiroshi Iida, Dung Thai, Varun Manjunatha, and Mohit Iyyer. 2021. TABBIE: Pretrained Representations of Tabular Data. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 3446–3456. doi:10.18653/v1/2021.naacl-main.270
- [23] Inwon Kang, Parikshit Ram, Yi Zhou, Horst Samulowitz, and Oshani Seneviratne. 2025. On Learning Representations for Tabular Data Distillation. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=GXlsrvOGIK>
- [24] Gjergji Kasneci and Enkelejda Kasneci. 2024. Enriching Tabular Data with Contextual LLM Embeddings: A Comprehensive Ablation Study for Ensemble Classifiers. *arXiv preprint arXiv:2411.01645* (2024). <http://arxiv.org/abs/2411.01645>
- [25] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems* 30 (2017).
- [26] Myung Jun Kim, Leo Grinsztajn, and Gael Varoquaux. 2024. CARTE: Pretraining and Transfer for Tabular Learning. In *Proceedings of the 41st International Conference on Machine Learning*. PMLR, 23843–23866. <https://proceedings.mlr.press/v235/kim24d.html>
- [27] Boshko Koloski, Andrei Margeloiu, Xiangjian Jiang, Blaž Škrj, Nikola Simidjievski, and Mateja Jamnik. 2025. LLM Embeddings for Deep Learning on Tabular Data. *arXiv preprint arXiv:2502.11596* (2025).
- [28] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. 2024. Table-GPT: Table Fine-tuned GPT for Diverse Table Tasks. In *Proc. ACM Manag. Data*, Vol. 2. 176:1–176:28. doi:10.1145/3654979
- [29] Jiaqi Mu and Pramod Viswanath. 2018. All-but-the-Top: Simple and Effective Postprocessing for Word Representations. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=HkuGJ3kCb>
- [30] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-Learn: Machine Learning in Python. 12, 85 (2011), 2825–2830. <http://jmlr.org/papers/v12/pedregosa11a.html>
- [31] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 3982–3992. doi:10.18653/v1/D19-1410
- [32] Marco Spinaci, Marek Polewicz, Johannes Hoffart, Markus C. Kohler, Sam Thelin, and Tassilo Klein. 2024. PORTAL: Scalable Tabular Foundation Models via Content-Specific Tokenization. *arXiv preprint arXiv:2410.13516* (2024).
- [33] Marco Spinaci, Marek Polewicz, Maximilian Schambach, and Sam Thelin. 2025. ConTextTab: A Semantics-Aware Tabular In-Context Learner. In *Advances in Neural Information Processing Systems*.
- [34] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [36] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace’s Transformers: State-of-the-Art Natural Language Processing. *arXiv preprint arXiv:1910.03771* (2020). <https://arxiv.org/abs/1910.03771>
- [37] Yuqian Wu, Hengyi Luo, and Raymond ST Lee. 2024. Deep feature embedding for tabular data. In *International Conference on Neural Information Processing*. Springer, 102–116.
- [38] Yazheng Yang, Yuqi Wang, Yaxuan Li, Sankalok Sen, Lei Li, and Qi Liu. 2025. Unleashing the Potential of Large Language Models for Predictive Tabular Tasks in Data Science. *arXiv preprint arXiv:2403.20208* (2025). <http://arxiv.org/abs/2403.20208>
- [39] Yazheng Yang, Yuqi Wang, Guang Liu, Ledell Wu, and Qi Liu. 2023. UniTabE: A Universal Pretraining Protocol for Tabular Foundation Model in Data Science. In *The Twelfth International Conference on Learning Representations*. [https://openreview.net/forum?id=6LLho5X6xV&referrer=%5Bthe%20profile%20of%20Yuqi%20Wang%5D\(%2Fprofile%3Fid%3D-Yuqi_Wang5\)](https://openreview.net/forum?id=6LLho5X6xV&referrer=%5Bthe%20profile%20of%20Yuqi%20Wang%5D(%2Fprofile%3Fid%3D-Yuqi_Wang5))
- [40] Yibo Zeng, Jiashuo Liu, Henry Lam, and Hongseok Namkoong. 2024. LLM Embeddings Improve Test-time Adaptation to Tabular $Y|X$ -Shifts. *arXiv preprint arXiv:2410.07395* (2024).

- [41] Li Zhang, Shuo Zhang, and Krisztian Balog. 2019. Table2Vec: Neural Word and Entity Embeddings for Table Population and Retrieval. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'19)*. Association for Computing Machinery, New York, NY, USA, 1029–1032. doi:10.1145/3331184.3331333
- [42] Yanlin Zhang, Ning Li, Quan Gan, Weinan Zhang, David Wipf, and Minjie Wang. 2024. ELF-Gym: Evaluating Large Language Models Generated Features for Tabular Prediction. *arXiv preprint arXiv:2410.12865* (2024). <http://arxiv.org/abs/2410.12865>
- [43] Lei Zheng, Ning Li, Xianyu Chen, Quan Gan, and Weinan Zhang. 2023. Dense Representation Learning and Retrieval for Tabular Data Prediction. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, Long Beach CA USA, 3559–3569. doi:10.1145/3580305.3599305

A Appendix

A.1 Reproducing Results in Paper

An anonymized version of our source code can be found in <https://anonymous.4open.science/r/tarlm-supplementary-F125/>. We will release the full code upon publication.

A.1.1 Implementation Details. All experiments were conducted on a single NVIDIA H100 GPU. We use the Hugging Face Transformers library [36] for all LLM-related computations. For the baseline models, we use the implementations from scikit-learn [30] and XGBoost [9].

A.2 Dataset Statistics

As noted in the main text, we use every dataset from the CARTE benchmark [26], and all but 4 datasets from the TabArena benchmark [12]. The 4 excluded datasets were omitted due to their large size, which made them impractical for our few-shot experiments which requires repeated sampling of small subsets.

A.3 Detailed results

Here we provide the results for regression tasks in addition to the classification results presented in the main text.

A.4 Meta-Learner implementation details

Algorithm 2 Meta-Training Dataset Generation

```

1: Input: Meta-training datasets  $\mathcal{D}_{\text{meta-train}}$ , support sizes  $\mathcal{K}$ , tasks per
   setting  $T$ , search space  $\Gamma_{\text{search}}$ , feature extractor  $f$ .
2: Initialize: Meta-datasets  $\{\mathcal{X}_k, \mathcal{Y}_k\}_{k \in \mathcal{K}}$  as empty.
3: for each dataset  $D \in \mathcal{D}_{\text{meta-train}}$  and each size  $k \in \mathcal{K}$  do
4:   Sample  $T$  tasks  $\{(S_i, (x_{q,i}, y_{q,i}))\}_{i=1}^T$  from  $D$ , where  $|S_i| = k$ .
5:   for each task  $i = 1, \dots, T$  do # Find optimal  $\gamma_i^*$  and construct meta-
     training pair
6:      $\gamma_i^* \leftarrow \arg \max_{\gamma \in \Gamma_{\text{search}}} \mathbb{I}(\text{TARL-PREDICT}(S_i, x_{q,i}, \gamma) = y_{q,i})$ 
7:      $z_i \leftarrow f(S_i)$ ; Append  $(z_i, \gamma_i^*)$  to  $(\mathcal{X}_k, \mathcal{Y}_k)$ . # extract features
     from context
8:   end for
9: end for
10: Return: Meta-datasets  $\{\mathcal{X}_k, \mathcal{Y}_k\}_{k \in \mathcal{K}}$ 

```

Algorithm 3 Meta-Inference with Learned Gamma Calibration

```

1: Input: Meta-datasets  $\{\mathcal{X}_k, \mathcal{Y}_k\}_{k \in \mathcal{K}}$ , a new task  $(S_{\text{test}}, x_{q,\text{test}})$  with
    $|S_{\text{test}}| = k$ , feature extractor  $f$ .
2: Train a regression model  $M_k$  on  $(\mathcal{X}_k, \mathcal{Y}_k)$ .
3: Compute heuristic  $\gamma_{\text{NN}}$  for the test task using Eq. 5.
4:  $z_{\text{test}} \leftarrow f(S_{\text{test}})$  # Extract same features as  $\mathcal{X}_k$ 
5:  $\tilde{\gamma} \leftarrow M_k(z_{\text{test}})$  # Predict the corrective scale factor
6:  $\hat{\gamma}_q \leftarrow \text{TARL-PREDICT}(S_{\text{test}}, x_{q,\text{test}}, \tilde{\gamma})$ 
7: Return: Prediction  $\hat{y}_q$ 

```

Table 8 lists the meta-features used for gamma prediction. These features are computed from the context set embeddings and labels, as well as the query embedding. We use the same feature set for all values of k (number of shots). The features can be categorized into three groups:

- **Basic Context Statistics:** These features capture fundamental properties of the context set, such as the number of classes and class distribution.
- **Context-Query Similarity Distribution Features:** These features describe the distribution of similarities and distances between the query example and the context examples, as well as between context examples and the mean context embedding.
- **γ -probing Features:** These features are derived from the model’s confidence scores (logits) when using a heuristic gamma value. They include the confidence for the majority and minority classes, as well as the entropy of the confidence distribution.

For the similarity-based features where we end up with a *distribution* of values (e.g., cosine similarities between the query and each context example), we compute summary statistics by computing the following: mean, standard deviation, minimum, maximum, skewness and kurtosis.

B Future Directions

While the primary goal of this work is to demonstrate the viability and efficiency of TaRL, our findings can be extended in several directions. We focus on binary classification tasks in this work, but our approach can be extended to multiclass classification and regression tasks. However, further investigation is needed to fully understand the limitations and potential of our approach in these contexts. In addition, our results with the γ parameter invite further analysis into what properties of a task determine its optimal calibration. Similarly, the success of a simple geometric correction via the CCR method motivates a more fine-grained analysis of how different tabular data structures are represented in the embedding space and whether there exist other methods that could further uncover the predictive signal in LLM embeddings.

Acknowledgments

This work was supported by IBM through the IBM-Rensselaer Future of Computing Research Collaboration.

Table 6: Statistics of datasets used in the experiments.

	Dataset	Benchmark	# classes	# rows	# cat. features	# cont. features
0	chocolate-bar-ratings	CARTE Binary	2	2588	6	2
1	coffee-ratings	CARTE Binary	2	2077	8	0
2	micelin	CARTE Binary	2	6832	5	2
3	nba-draft	CARTE Binary	2	1669	4	2
4	ramen-ratings	CARTE Binary	2	4084	4	0
5	roger-ebert	CARTE Binary	2	2687	6	1
6	spotify	CARTE Binary	2	41099	6	11
7	us-accidents-severity	CARTE Binary	2	21656	7	1
8	whisky	CARTE Binary	2	1812	6	0
9	yelp	CARTE Binary	2	64616	8	3
10	zomato	CARTE Binary	2	61441	7	1
11	blood-transfusion-service-center	TabArena Binary	2	748	0	4
12	Amazon-employee-access	TabArena Binary	2	32769	9	0
13	APSFailure	TabArena Binary	2	76000	1	168
14	bank-marketing	TabArena Binary	2	45211	5	5
15	Bank-Customer-Churn	TabArena Binary	2	10000	3	4
16	anneal	TabArena Multiclass	5	898	19	3
17	credit-card-clients-default	TabArena Binary	2	30000	8	14
18	customer-satisfaction-in-airline	TabArena Binary	2	129880	15	4
19	coil2000-insurance-policies	TabArena Binary	2	9822	80	0
20	diabetes	TabArena Binary	2	768	1	7
21	credit-g	TabArena Binary	2	1000	14	3
22	Diabetes130US	TabArena Binary	2	71518	36	3
23	churn	TabArena Binary	2	5000	3	14
24	GiveMeSomeCredit	TabArena Binary	2	150000	4	6
25	hazelnut-spread-contaminant-detection	TabArena Binary	2	2400	0	30
26	E-CommerceShippingData	TabArena Binary	2	10999	6	3
27	heloc	TabArena Binary	2	10459	6	17
28	Fitness-Club	TabArena Binary	2	1500	3	2
29	HR-Analytics-Job-Change-of-Data-Scientists	TabArena Binary	2	19158	9	2
30	in-vehicle-coupon-recommendation	TabArena Binary	2	12684	17	0
31	Is-this-a-good-customer	TabArena Binary	2	1723	6	4
32	maternal-health-risk	TabArena Multiclass	3	1014	4	2
33	online-shoppers-intention	TabArena Binary	2	12330	8	8
34	Marketing-Campaign	TabArena Binary	2	2240	9	10
35	kddcup99-appetency	TabArena Binary	2	50000	77	123
36	NATIOUSdroid	TabArena Binary	2	7491	0	0
37	students-dropout-and-academic-success	TabArena Multiclass	3	4424	16	12
38	polish-companies-bankruptcy	TabArena Binary	2	5910	0	64
39	seismic-bumps	TabArena Binary	2	2584	6	6
40	qsar-biodeg	TabArena Binary	2	1054	19	19
41	SDSS17	TabArena Multiclass	3	78053	3	8
42	splice	TabArena Multiclass	3	3190	60	0
43	website-phishing	TabArena Multiclass	3	1353	7	0
44	MIC	TabArena Multiclass	8	1699	93	11
45	jml	TabArena Binary	2	10885	0	21
46	anime-planet	CR	1	14391	5	4
47	babies-r-us	CR	1	5085	3	0
48	beer-ratings	CR	1	3197	5	14
49	bikedekho	CR	1	4786	5	2
50	bikewale	CR	1	8992	5	2
51	buy-buy-baby	CR	1	10718	3	0
52	cardekho	CR	1	37813	12	2
53	clear-corpus	CR	1	4724	12	13
54	company-employees	CR	1	10941	7	0
55	employee-remuneration	CR	1	35396	2	2
56	employee-salaries	CR	1	9211	4	2
57	fifa22-players	CR	1	18085	9	8
58	filmtv-movies	CR	1	41205	6	3
59	journal-jcr	CR	1	9615	4	5
60	journal-sjr	CR	1	27931	9	0
61	jp-anime	CR	1	15535	10	5
62	k-drama	CR	1	1239	8	4
63	mlds-salaries	CR	1	10456	7	1
64	movies	CR	1	7224	7	6
65	museums	CR	1	11467	14	2
66	mydramalist	CR	1	3400	10	3
67	prescription-drugs	CR	1	1714	5	2
68	rotten-tomatoes	CR	1	7158	9	4
69	us-accidents-counts	CR	1	22623	6	0
70	us-presidential	CR	1	19857	6	0
71	used-cars-24	CR	1	5918	5	2
72	used-cars-benz-italy	CR	1	16391	2	3
73	used-cars-dot-com	CR	1	4009	7	2
74	used-cars-pakistan	CR	1	72655	2	3
75	used-cars-saudi-arabia	CR	1	5507	6	3
76	videogame-sales	CR	1	16410	4	1
77	wikiliq-beer	CR	1	13461	7	2
78	wikiliq-spirit	CR	1	12275	5	2
79	wina-pl	CR	1	2247	11	2
80	wine-dot-com-prices	CR	1	15254	7	2
81	wine-dot-com-ratings	CR	1	4095	7	2
82	wine-enthusiasts-prices	CR	1	120975	8	1
83	wine-enthusiasts-ratings	CR	1	129971	8	1
84	wine-vivino-price	CR	1	13834	5	2
85	wine-vivino-rating	CR	1	13834	6	2
86	airfoil-self-noise	TR	1	1503	3	2
87	Another-Dataset-on-used-Fiat-500	TR	1	1538	3	4
88	concrete-compressive-strength	TR	1	1030	1	7
89	diamonds	TR	1	53940	3	6
90	healthcare-insurance-expenses	TR	1	1338	2	2
91	Food-Delivery-Time	TR	1	45451	3	6
92	houses	TR	1	20640	0	8
93	miami-housing	TR	1	13776	2	12
94	physiochemical-protein	TR	1	45730	0	9
95	QSAR-fish-toxicity	TR	1	907	2	4
96	superconductivity	TR	1	21263	2	79
97	wine-quality	TR	1	6497	0	11

Table 7: Detailed comparison of average balanced accuracy score per benchmark. Best performance in each benchmark/ k combination is marked with bold and the second best with underline.

Benchmark	Model	Rank		k (number of shots)							
		$k \leq 32$	$k > 32$	4	8	16	32	64	128	256	512
CARTE Regression (40)	KNN	6.8125	7.9562	0.2803	0.2814	0.2985	0.2985	0.3139	0.3223	0.3271	0.3355
	KNN-Emb.	7.2750	7.5000	0.2539	0.2756	0.2918	0.3076	0.3218	0.3469	0.3628	0.3766
	XGBoost	6.1625	5.7625	0.2437	0.3004	0.3441	0.3763	0.3997	0.4386	0.4637	0.4843
	TabPFN	4.3688	3.7687	0.3147	0.3237	0.3740	0.4174	0.4394	0.4831	0.5118	0.5472
	ConTextTab	<u>5.1125</u>	<u>4.1125</u>	<u>0.2993</u>	<u>0.3195</u>	<u>0.3537</u>	<u>0.3956</u>	<u>0.4278</u>	<u>0.4810</u>	<u>0.5100</u>	<u>0.5311</u>
	mTaRL	6.5563	7.4750	0.2767	0.2965	0.3078	0.3158	0.3151	0.3509	0.3710	0.3737
	Oracle	0.5844	1.1875	0.5021	0.5230	0.5393	0.5554	0.5759	0.5925	0.6097	0.6210
TabArena Multiclass (6)	KNN	7.0714	7.2143	0.2940	0.3388	0.3823	0.4031	0.4352	0.5199	0.5313	0.4930
	KNN-Emb.	7.7857	7.9286	0.2876	0.3594	0.4029	0.4363	0.4574	0.4972	0.5623	0.5511
	XGBoost	6.3214	3.9643	0.2190	0.4352	0.5262	0.6323	0.6667	0.7019	0.7516	0.7762
	TabPFN	4.3929	2.0893	0.3993	0.4677	0.5508	0.6521	0.7023	0.7567	0.7945	0.8109
	ConTextTab	3.9107	2.9821	0.3561	0.5242	0.5562	0.6709	0.7050	0.7686	0.7697	0.7692
	mTaRL	6.8750	7.6429	0.3263	0.4260	0.4218	0.4729	0.4961	0.5354	0.5863	0.5867
	Oracle	1.6964	3.6429	0.4402	0.6150	0.6366	0.6690	0.6781	0.6969	0.7455	0.7403
TabArena Regression (13)	KNN	6.0625	7.6042	0.2917	0.3410	0.3736	0.3858	0.3953	0.4404	0.4689	0.4745
	KNN-Emb.	7.6562	8.1875	0.2638	0.2829	0.2918	0.3238	0.3674	0.3911	0.4320	0.4671
	XGBoost	6.1667	5.0000	0.2292	0.3498	0.4318	0.4613	0.5110	0.5703	0.5978	0.6381
	TabPFN	3.2917	2.6667	0.3472	0.4042	0.4830	0.5340	<u>0.5843</u>	0.6005	0.6572	0.7011
	ConTextTab	<u>4.0000</u>	3.2917	0.3348	0.3906	0.4600	0.5200	0.5901	0.5845	0.6341	0.6624
	mTaRL	7.0417	7.8958	0.2715	0.3079	0.3509	0.3533	0.3987	0.4291	0.4457	0.4692
	Oracle	0.9583	1.8125	0.4836	0.5143	0.5570	0.5768	0.6289	0.6525	0.6619	0.6955

Table 8: Meta-features for Gamma Prediction in Few-shot Learning

Feature Name	Description
Basic Context Statistics	
num_classes	Number of unique classes.
minority_class_ratio	Fraction of minority class.
majority_class_ratio	Fraction of majority class.
Context-Query Similarity Distribution Features	
sim	Statistics of cosine similarity (normalized dot prod.) between query and context examples.
l2	Statistics of euclidean distance between query and context examples.
Context-Query Similarity Distribution Features	
sim-mean	Statistics of cosine similarity (normalized dot prod.) between context examples and mean context embedding.
l2-mean	Statistics of euclidean distance between context examples and mean context embedding.
γ-probing Features	
majority	Confidence for majority label.
minority	Confidence for minority label.
entropy	Entropy of confidence (logits).